

JAGIELLONIAN UNIVERSITY
IN KRAKOW

Lecture 6: Probability and Variational Autoencod
Latent Spaces, Conditional and Disentangled Generation
Digits and Blood Cells

Deep Learning for Scientific Discovery

Marcin Płodzień

Atomic Optics Department
Institute of Theoretical Physics
Jagiellonian University in Kraków



Doctoral School
of Exact and
Natural Sciences
Jagiellonian University

Recap: Understanding Data without Labels

- PCA and t-SNE: a few coordinates that describe each sample
- k-means and DBSCAN: groups of similar samples
- autoencoders: a learned compression into a latent space; the reconstruction error flags unusual samples
- anomaly detection: unusual blood cells and credit-card fraud flagged by a large reconstruction error

Today: learn the distribution of the data itself, and draw new samples from it.

1

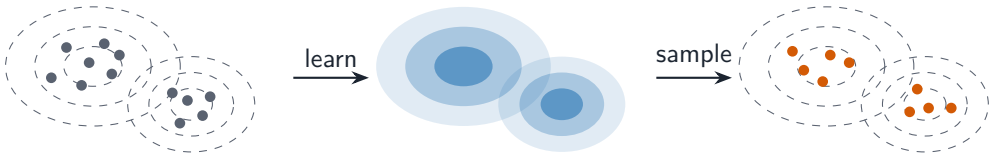
Generative Models

JAGIELLONIAN UNIVERSITY
IN KRAKOW

Learning a Probability Distribution

We have only data $\mathbf{x}_1, \ldots, \mathbf{x}_n$: images, spectra, spin configurations. No labels, nothing else. We assume they are independent samples of an unknown probability distribution $p(\mathbf{x})$.

Goal: a model $p_\theta(\mathbf{x})$ that we can *sample*: new $\mathbf{x} \sim p_\theta$ that look as if they came from $p(\mathbf{x})$ itself, with the same statistics (shapes, frequencies, correlations, energies), and that are not copies of the data.



data: samples of an unknown $p(\mathbf{x})$ (dashed)

model $p_\theta(\mathbf{x})$

new samples: where $p(\mathbf{x})$ puts its data

Supervised learning models $p(y \mid \mathbf{x})$; Lecture 5 found structure in the data; today we learn $p(\mathbf{x})$ itself.

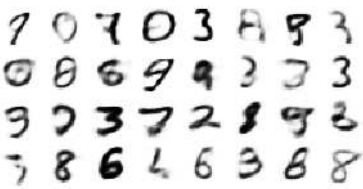
Generating New Data

A **generative model**

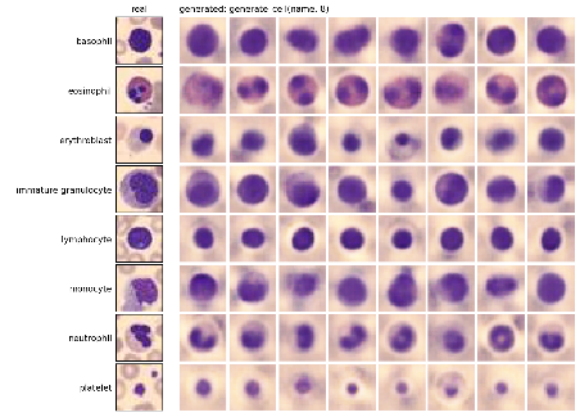
- learns an approximation $p_\theta(\mathbf{x})$ of $p(\mathbf{x})$ from the samples (θ: its parameters), by making the observed data probable under the model
- draws **new samples**: instances not in the data set, but that could have been
- can be checked: the generated samples must reproduce the statistics of the data (for spins, the physics: magnetisation, energy, correlations)

In science: new molecules and materials, protein structures, weather ensembles, fast surrogates of expensive simulations; and the learned p_θ itself as a description of the data.

Generated Digits and Blood Cells



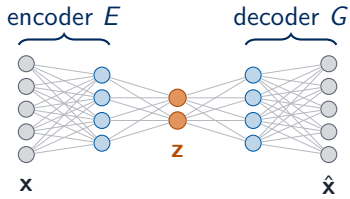
new digits: no image of the training set

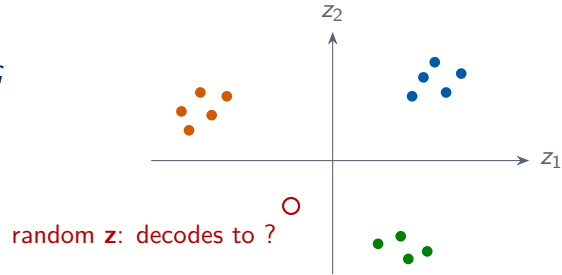


blood cells of a chosen type (first column: a real cell)

Both made by the models of this lecture, trained on a laptop CPU in minutes.

Autoencoder and Its Latent Space

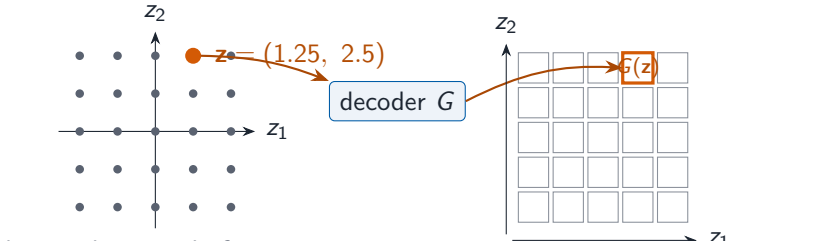




codes $E(\mathbf{x}_i)$ of training images (colour: digit)

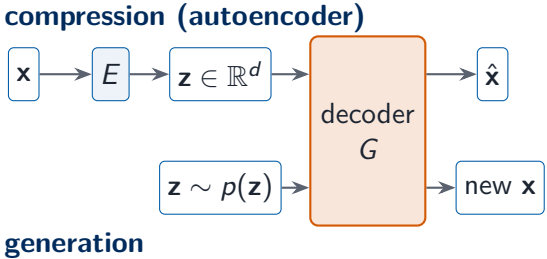
- trained only to rebuild its inputs: each image becomes one point $\mathbf{z}_i = E(\mathbf{x}_i)$
- nothing fixes where the points lie: no known distribution to draw a new $\mathbf{z}$ from
- the decoder never saw the gaps: a $\mathbf{z}$ there decodes to something that is no digit

Latent Grid



- a regular grid of latent points $\mathbf{z} = (z_1, z_2)$, $z_1, z_2 \in [-2.5, 2.5]$; each is decoded and its image placed at the same position (column: z_1, row: z_2)
- the sheet is a map of what the decoder produces everywhere in the latent plane, also where no training image was mapped

Compression and Generative Modelling



store each $\mathbf{x}_i$ in d numbers:

$$\min_{E,G} \frac{1}{N} \sum_{i=1}^{N} \|\mathbf{x}_i - G(E(\mathbf{x}_i))\|^2$$

new $\mathbf{x}$ distributed like the data:

$$p_\theta(\mathbf{x}) = \int p_\theta(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})\, d\mathbf{z} \approx p_{\text{data}}(\mathbf{x})$$

E: encoder; N: number of training images $\mathbf{x}_i$; θ: weights of the decoder; $p_\theta(\mathbf{x} \mid \mathbf{z})$: the probability it assigns to $\mathbf{x}$ given $\mathbf{z}$; p_{data}: the distribution of the data.

- generation reuses the decoder of compression; missing are (i) a known $p(\mathbf{z})$ to draw from, (ii) a decoder trained for every $\mathbf{z}$ that $p(\mathbf{z})$ produces, not only the N codes
- **the VAE adds both:** each code becomes a Gaussian cloud; a KL term (a distance between distributions) pulls the clouds towards $\mathcal{N}(0, I)$

Compression and Generation Compared

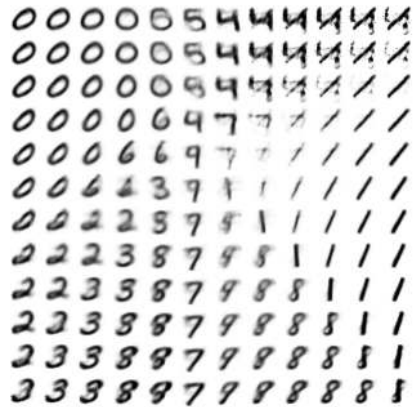
	compression (autoencoder)	generation
input	an image $\mathbf{x}$	a random $\mathbf{z} \sim p(\mathbf{z})$
output	$\hat{\mathbf{x}} = G(E(\mathbf{x})) \approx \mathbf{x}$	a new $\mathbf{x}$, not in the training set
objective	reconstruction error $\frac{1}{N}\sum_i \|\mathbf{x}_i - G(E(\mathbf{x}_i))\|^2$	$p_\theta(\mathbf{x}) \approx p_{\text{data}}(\mathbf{x})$
learned	encoder E and decoder G	decoder G; $p(\mathbf{z})$ chosen, $\mathcal{N}(0, I)$
latent codes	N points $\mathbf{z}_i = E(\mathbf{x}_i)$	a density $p(\mathbf{z})$ over a whole region
use	storage, denoising, features for other tasks	new samples: data augmentation, simulation

An Autoencoder Cannot Generate

Can the decoder of an autoencoder produce *new* digits? Pick a latent point $\mathbf{z}$ and compute $G(\mathbf{z})$.

- the autoencoder only learned to rebuild its training images; nothing tells us where in latent space the valid $\mathbf{z}$ are
- between and around the training codes there are regions that decode to images that are no digit

The sheet shows what the decoder does where it was never trained.



autoencoder with $d = 2$: images decoded from a regular grid of latent points

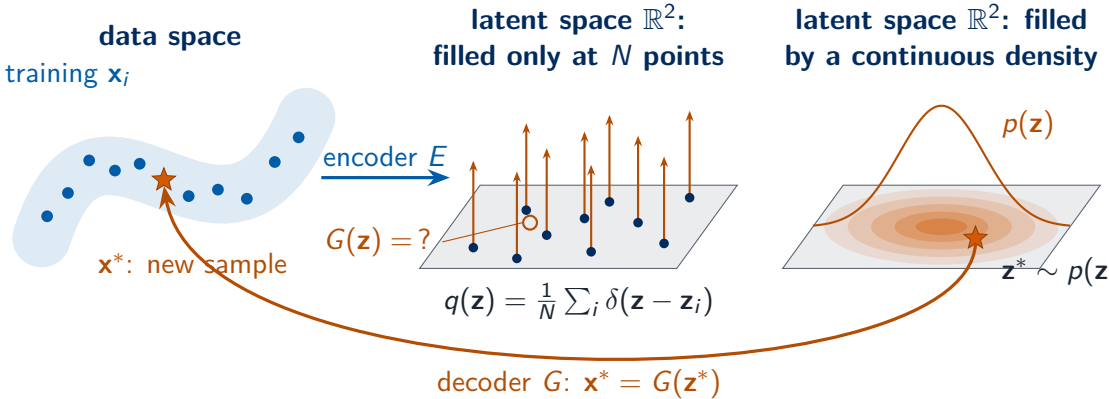
A Finite Set of Latent Codes

1. the latent space $\mathbb{R}^d$ (for $d = 2$ the plane (z_1, z_2)) is continuous: every $\mathbf{z} \in \mathbb{R}^d$ is a valid input of the decoder, $G(\mathbf{z})$ is defined
2. the autoencoder fills it only at N points, one per training image: $\mathbf{z}_i = E(\mathbf{x}_i)$, $i = 1, \ldots, N$ (MNIST: $N = 60\,000$ coloured dots)
3. the decoder was trained only at these points, so the only latent distribution the autoencoder provides is a sum of spikes,

$$q(\mathbf{z}) = \frac{1}{N} \sum_{i=1}^{N} \delta(\mathbf{z} - \mathbf{z}_i), \qquad \delta : \text{Dirac delta, a spike at } \mathbf{0} \text{ with } \int \delta(\mathbf{z})\, d\mathbf{z} = 1$$

   a draw from q returns some $\mathbf{z}_i$, and $G(\mathbf{z}_i) \approx \mathbf{x}_i$ rebuilds a training image: nothing new. For any other $\mathbf{z}$, $G(\mathbf{z})$ is unconstrained
4. wanted instead: a continuous density $p(\mathbf{z}) > 0$ over a whole region, with the decoder trained on all of it; the VAE turns each dot into a Gaussian cloud

Discrete Codes and a Continuous Latent Density



Middle: one spike per code $\mathbf{z}_i = E(\mathbf{x}_i)$, untrained gaps between them. Right: a draw from a density decodes to a new sample between the training images.

Generation with a Latent Space

The goal: new instances $\mathbf{x}$, drawn from (an approximation of) the distribution $p(\mathbf{x})$ of the data. The recipe of a latent-variable model:

$$\underbrace{\mathbf{z} \sim p(\mathbf{z})}_{\text{a random latent point}} \quad \longrightarrow \quad \underbrace{\mathbf{x} = G(\mathbf{z})}_{\text{decode it}}$$

It works only if two things hold:

1. $p(\mathbf{z})$ is *known* and easy to sample: we choose $p(\mathbf{z}) = \mathcal{N}(0, I)$
2. *every* $\mathbf{z}$ that $p(\mathbf{z})$ produces decodes to a plausible $\mathbf{x}$: the training data must fill the region where $p(\mathbf{z})$ puts its probability, without gaps

A finite set of codes satisfies neither. The variational autoencoder is built to satisfy both; its decoder becomes this generator G, and the model of the data is $p_\theta(\mathbf{x}) = \int p_\theta(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})\, d\mathbf{z}$, which should approximate $p_{\text{data}}(\mathbf{x})$.

2
Probability



JAGIELLONIAN UNIVERSITY
IN KRAKOW

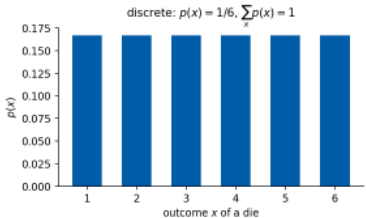
Discrete and Continuous Distributions

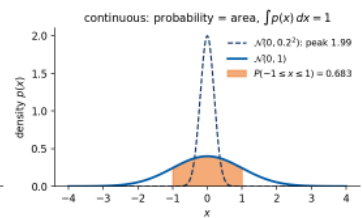
Discrete x (a die, a class, a spin): $p(x)$ is a probability,

$$0 \le p(x) \le 1, \qquad \sum_x p(x) = 1.$$

Continuous x (an intensity, a coordinate): $p(x)$ is a *density*; only areas are probabilities,

$$P(a \le x \le b) = \int_a^b p(x)\,dx, \qquad \int p(x)\,dx = 1$$



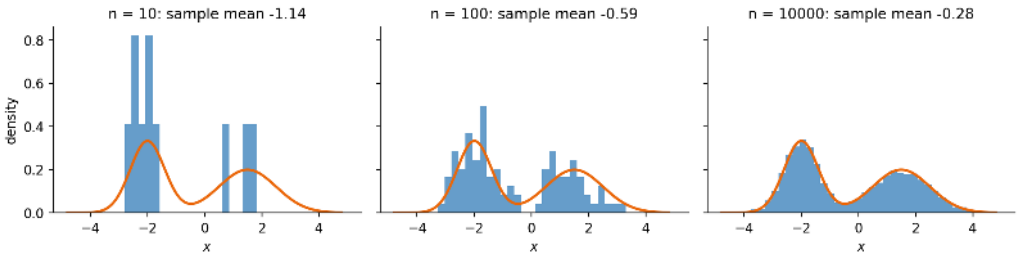


A density can exceed 1 (peak 1.99 for $\sigma = 0.2$). An MNIST image is a point $\mathbf{x} \in \mathbb{R}^{784}$, and $p(\mathbf{x})$ is a density in 784 dimensions.

Expectation and Averages over Samples

The **expectation** of a function f under p, and its estimate from samples $x_1, \ldots, x_n$ of p:

$$\mathbb{E}_{x\sim p}[f(x)] = \int p(x)\, f(x)\, dx \;\approx\; \frac{1}{n}\sum_{i=1}^{n} f(x_i), \qquad \text{std of the estimate} = \frac{\sigma_f}{\sqrt{n}}$$



Orange: $p(x) = \frac{1}{2}\mathcal{N}(x; -2, 0.6^2) + \frac{1}{2}\mathcal{N}(x; 1.5, 1)$, with $\mathbb{E}[x] = -0.25$; blue: histograms of n samples. Every loss in this lecture is an expectation, estimated as an average over a mini-batch (σ_f: standard deviation of $f(x)$ under p).

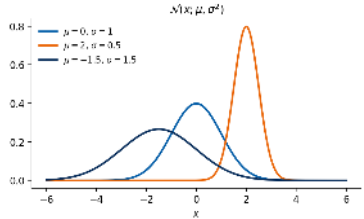
The Gaussian Distribution

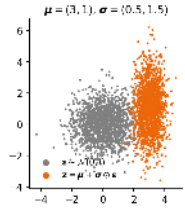
One dimension, mean μ, standard deviation σ:

$$\mathcal{N}(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(x-\mu)^2/2\sigma^2}$$

d independent coordinates; $\mathcal{N}(0, I)$: all $\mu_k = 0$, $\sigma_k = 1$:

$$\mathcal{N}(\mathbf{z}; \boldsymbol{\mu}, \operatorname{diag} \boldsymbol{\sigma}^2) = \prod_{k=1}^{d} \mathcal{N}(z_k; \mu_k, \sigma_k^2)$$



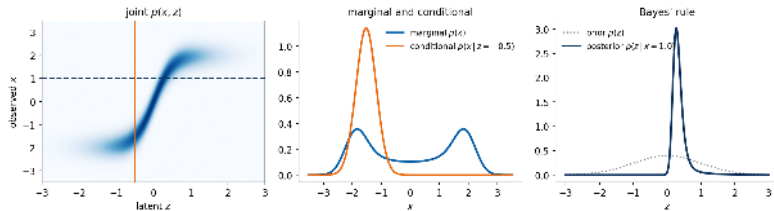


Sampling: draw $\varepsilon \sim \mathcal{N}(0, I)$, then $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \varepsilon$ ($\odot$: element-wise product) is a sample of $\mathcal{N}(\boldsymbol{\mu}, \operatorname{diag} \boldsymbol{\sigma}^2)$: a shift and a stretch.

Joint, Conditional and Marginal Distributions

An observed x and a hidden (latent) z, with joint distribution $p(x, z)$:

$$\underbrace{p(x) = \int p(x, z)\, dz}_{\text{marginal}} \qquad \underbrace{p(x, z) = p(x \mid z)\, p(z)}_{\text{conditional, product rule}} \qquad \underbrace{p(z \mid x) = \frac{p(x \mid z)\, p(z)}{p(x)}}_{\text{Bayes' rule: posterior}}$$



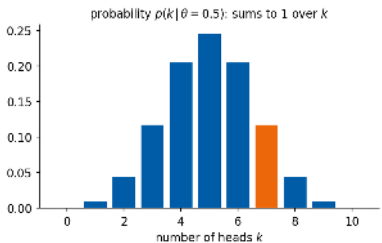
$p(z)$: the *prior*, what we know about z before seeing x; $p(z \mid x)$: the *posterior*, after seeing x. Here $z \sim \mathcal{N}(0, 1)$, $x \mid z \sim \mathcal{N}\big(2 \tanh(2z), 0.35^2\big)$: a two-peaked $p(x)$; observing $x = 1$ narrows the prior (std 1) to a posterior with std 0.25.

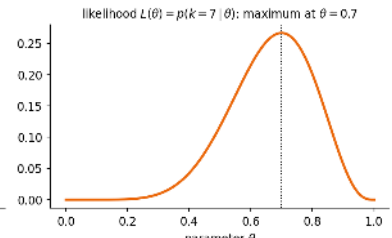
Probability and Likelihood

A model $p(x \mid \theta)$: the probability of data x when the parameters are θ. One formula, read in two ways:

probability: θ fixed, x varies; summed over all outcomes x it gives 1

likelihood $L(\theta) = p(x_{\text{obs}} \mid \theta)$: data fixed, θ varies; not a distribution over θ





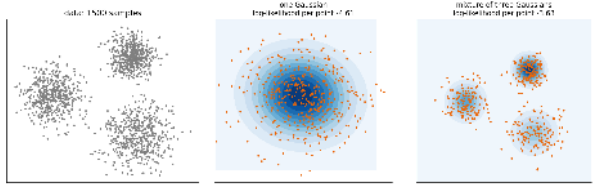
A coin with probability θ of heads, tossed 10 times: $p(k \mid \theta) = \binom{10}{k}\theta^k(1-\theta)^{10-k}$; $p(7 \mid 0.5) = 0.117$. With 7 heads observed, $L(\theta)$ peaks at $\theta = 0.7$, and $\int_0^1 L\, d\theta = 1/11$, not 1.

Maximum Likelihood

Independent samples $\mathbf{x}_1, \ldots, \mathbf{x}_n$; a model $p_\theta(\mathbf{x})$. The likelihood of all the data, and its logarithm:

$$L(\theta) = \prod_{i=1}^{n} p_\theta(\mathbf{x}_i), \qquad \ell(\theta) = \sum_{i=1}^{n} \log p_\theta(\mathbf{x}_i), \qquad \theta^* = \arg\max_\theta \ell(\theta)$$

The logarithm turns the product into a sum and does not move the maximum. One Gaussian: θ^* is the sample mean and variance (with $1/n$).



1500 points from three groups; blue: fitted density, orange: 400 new samples. Mean log-likelihood per point: -4.61 (one Gaussian), -3.63 (mixture of three).

The Kullback–Leibler Divergence

Mismatch of a model q with the true p:

$$\mathrm{KL}(p \,\|\, q) = \mathbb{E}_{x\sim p}\Big[\log \frac{p(x)}{q(x)}\Big]$$

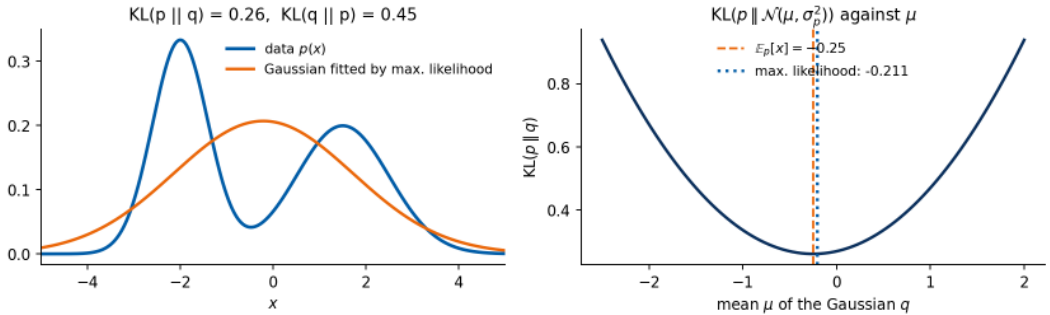
≥ 0, $= 0$ only if $q = p$, not symmetric (so not a distance). With $-\log q(x)$ the surprise of seeing x under q: the average extra surprise when data from p are described by q.

With $q = p_\theta$, split the logarithm:

$$\mathrm{KL}(p \,\|\, p_\theta) = \underbrace{\mathbb{E}_p[\log p]}_{\text{no } \theta} - \underbrace{\mathbb{E}_p[\log p_\theta]}_{\approx \ell(\theta)/n}$$

maximum likelihood = minimum $\mathrm{KL}(p \,\|\, p_\theta)$

Maximum Likelihood and the KL Divergence

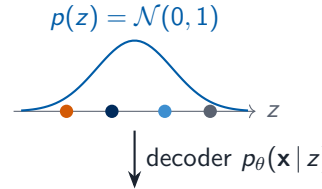


One Gaussian (width fixed at σ_p, the std of p) fitted to the two-peaked p: the KL is smallest at $\mu = \mathbb{E}_p[x] = -0.25$; maximum likelihood on 10^4 samples gives -0.21.

Latent-Variable Models

A simple hidden $\mathbf{z}$ and a map to the data:

$$p_\theta(\mathbf{x}) = \int p_\theta(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})\, d\mathbf{z}$$



- **generate**: draw $\mathbf{z} \sim p(\mathbf{z})$, then $\mathbf{x} \sim p_\theta(\mathbf{x} \mid \mathbf{z})$
- Gaussian mixture: $z \in \{1, \ldots, K\}$ with $p(z = k) = \pi_k$, $p(\mathbf{x} \mid z = k) = \mathcal{N}(\boldsymbol{\mu}_k, \Sigma_k)$
- VAE: $\mathbf{z} \in \mathbb{R}^d$, $p(\mathbf{z}) = \mathcal{N}(0, I)$, and a neural network (the decoder) sets $p_\theta(\mathbf{x} \mid \mathbf{z})$

each z gives a small cloud around a point of a curve in data space

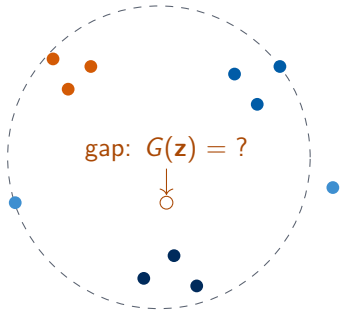
3

Variational Autoencoders

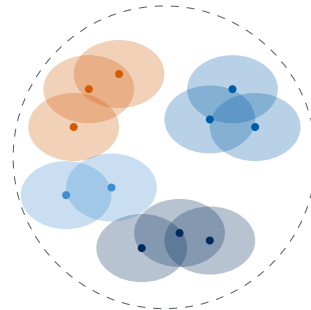
JAGIELLONIAN UNIVERSITY
IN KRAKÓW

Latent Points and Latent Clouds

autoencoder: one point per input



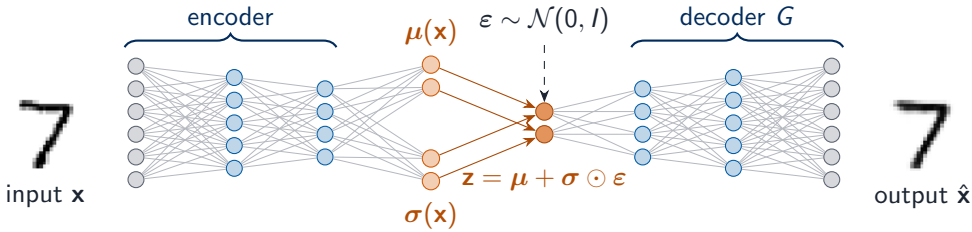
VAE: a Gaussian cloud per input



dashed circle: where $\mathcal{N}(0, I)$ puts most of its probability

Colours: four kinds of input. For each input $\mathbf{x}$ the VAE encoder outputs a centre $\boldsymbol{\mu}(\mathbf{x})$ and widths $\boldsymbol{\sigma}(\mathbf{x})$: the cloud $\mathcal{N}(\boldsymbol{\mu}(\mathbf{x}), \operatorname{diag} \boldsymbol{\sigma}^2(\mathbf{x}))$. Neighbouring clouds overlap, and together they cover the dashed circle.

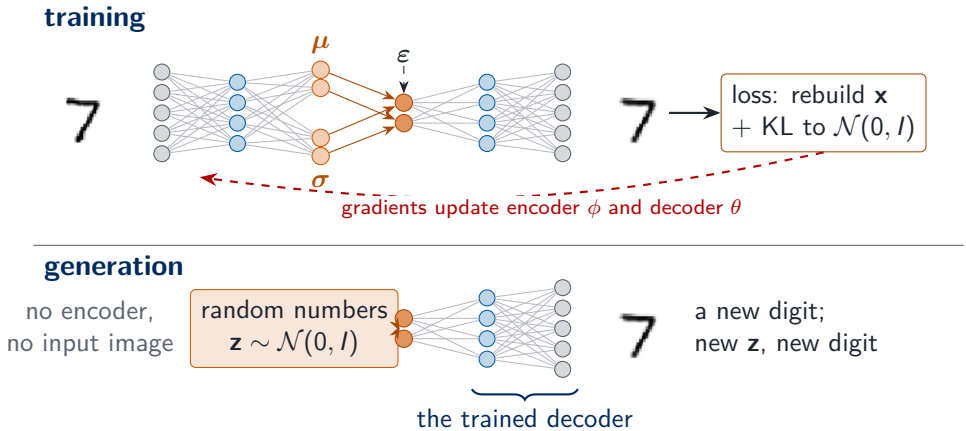
Variational Autoencoder



- an autoencoder maps an input to one *point* $\mathbf{z}$; a VAE maps it to a Gaussian *cloud* centred at $\mu(\mathbf{x})$ with widths $\sigma(\mathbf{x})$ (one μ_k, σ_k for each of the d latent coordinates)
- during training a random point of the cloud is decoded, so every point near $\mu(\mathbf{x})$ must decode to something like $\mathbf{x}$
- to *generate*, skip the encoder: draw $\mathbf{z} \sim \mathcal{N}(0, I)$ and compute $G(\mathbf{z})$

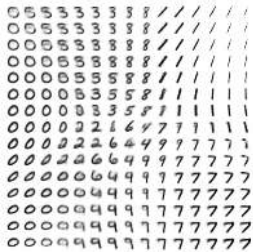
D. P. Kingma, M. Welling, *Auto-Encoding Variational Bayes*, ICLR 2014, arXiv:1312.6114

Training and Sampling a VAE

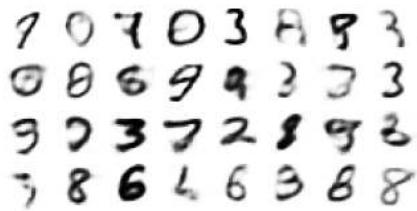


The KL term has placed the training clouds inside $\mathcal{N}(0, I)$, so random numbers drawn from it land where the decoder has learned to produce digits.

New Digits from the VAE



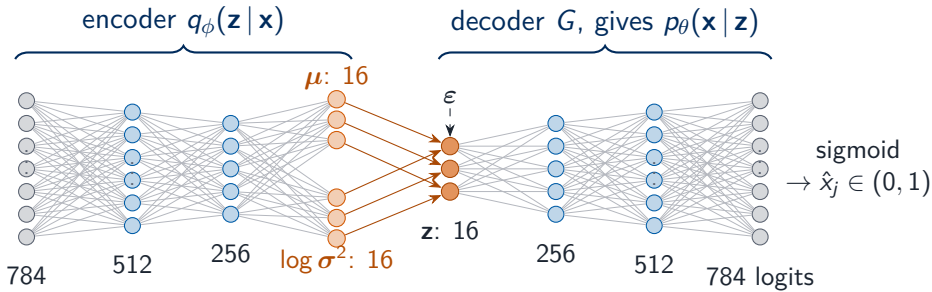
$d = 2$: images decoded from a grid of $\mathbf{z}$ over $[-2.5, 2.5]^2$



$d = 16$: 32 new digits, each from a random $\mathbf{z} \sim \mathcal{N}(0, I)$

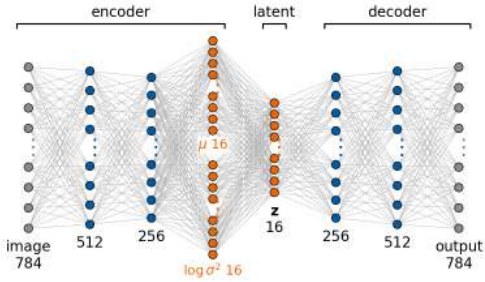
- continuous latent space: neighbouring $\mathbf{z}$ give similar digits
- new, not memorised: on average a generated digit is as far from its nearest training image (Euclidean distance in pixel space, 4.73) as a real test digit the network never saw (4.03); somewhat blurry, a known property of VAEs

The VAE on MNIST: Layers



Dense layers with ReLU (a few nodes drawn per layer; the numbers give the true sizes); logits: the outputs before the sigmoid. ϕ: weights of the encoder, θ: of the decoder. The encoder outputs $\log \sigma^2$, which can take any sign; $d = 2$ or 16. Trained 20 epochs with Adam on the 60 000 training digits, without labels.

VAE Layers and Parameters



layer	output	param.
encoder, input $\mathbf{x}$*: 784*		
dense + ReLU	512	401 920
dense + ReLU	256	131 328
dense $\to \boldsymbol{\mu}$	16	4 112
dense $\to \log \boldsymbol{\sigma}^2$	16	4 112
decoder, input $\mathbf{z}$*: 16*		
dense + ReLU	256	4 352
dense + ReLU	512	131 584
dense (logits)	784	402 192

$d = 16$ latent coordinates; a dense layer from n to m nodes has $nm + m$ parameters. Total 1 079 600 parameters, all trainable (torchinfo).

The Decoder Defines $p_\theta(\mathbf{x} \mid \mathbf{z})$

The decoder ends with a sigmoid, $\hat{\mathbf{x}} = G(\mathbf{z}) \in (0,1)^{784}$ (its last layer outputs *logits*, the numbers before the sigmoid); $\hat{x}_j$ is the probability that pixel j is ink. For black/white pixels $x_j \in \{0,1\}$, drawn independently given $\mathbf{z}$:

$$p_\theta(\mathbf{x} \mid \mathbf{z}) = \prod_{j=1}^{784} \hat{x}_j^{\,x_j} \, (1-\hat{x}_j)^{1-x_j} \quad \Longrightarrow \quad -\log p_\theta(\mathbf{x} \mid \mathbf{z}) = -\sum_{j=1}^{784} \Big[x_j \log \hat{x}_j + (1-x_j) \log(1-\hat{x}_j) \Big]$$

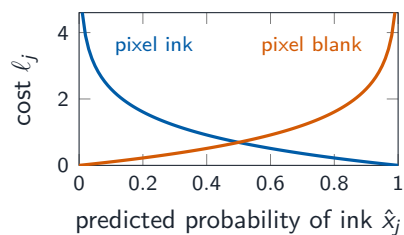
the reconstruction loss (binary cross-entropy) is minus a log-likelihood. MNIST pixels are grey, $x_j \in [0,1]$: the same formula is still used as the loss.

Binary Cross-Entropy

Pixel j is ink ($x_j = 1$) or blank ($x_j = 0$); the decoder gives the probability of ink $\hat{x}_j$. The cost of pixel j is minus the log-probability given to what it actually is; the reconstruction loss is $\sum_{j=1}^{784} \ell_j$:

$$\ell_j = -\big[x_j \log \hat{x}_j + (1 - x_j) \log(1 - \hat{x}_j)\big] = \begin{cases} -\log \hat{x}_j, & x_j = 1 \text{ (ink)} \\ -\log(1 - \hat{x}_j), & x_j = 0 \text{ (blank)} \end{cases}$$

- $\ell_j = 0$ only if the decoder is certain and right; it grows without bound if it is confident and wrong
- *binary*: two outcomes per pixel; *cross-entropy*: the surprise $-\log$ of the true outcome (classification loss, two classes)



Gaussian Pixels and Independent Pixels

Gaussian pixels give the squared error

$p_\theta(\mathbf{x} \mid \mathbf{z}) = \mathcal{N}(\mathbf{x}; G(\mathbf{z}), \sigma_x^2 I)$:

$$-\log p_\theta = \frac{\|\mathbf{x} - G(\mathbf{z})\|^2}{2\sigma_x^2} + \text{const}$$

Independent pixels

given $\mathbf{z}$, the product treats pixels as unrelated: all correlations between pixels must come from $\mathbf{z}$.

The Evidence Lower Bound

Maximum likelihood needs $\log p_\theta(\mathbf{x}) = \log \int p_\theta(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})\, d\mathbf{z}$: an integral over all $\mathbf{z}$, out of reach. The encoder $q_\phi(\mathbf{z} \mid \mathbf{x})$ approximates the posterior $p_\theta(\mathbf{z} \mid \mathbf{x})$. Since $p_\theta(\mathbf{x}) = p_\theta(\mathbf{x}, \mathbf{z}) / p_\theta(\mathbf{z} \mid \mathbf{x})$ for every $\mathbf{z}$, average log of it over $\mathbf{z} \sim q_\phi$ and multiply inside by q_ϕ / q_ϕ:

$$\log p_\theta(\mathbf{x}) = \mathbb{E}_{q_\phi}\Big[\log \frac{p_\theta(\mathbf{x}, \mathbf{z})}{q_\phi(\mathbf{z} \mid \mathbf{x})}\Big] + \mathbb{E}_{q_\phi}\Big[\log \frac{q_\phi(\mathbf{z} \mid \mathbf{x})}{p_\theta(\mathbf{z} \mid \mathbf{x})}\Big]$$

With $p_\theta(\mathbf{x}, \mathbf{z}) = p_\theta(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})$:

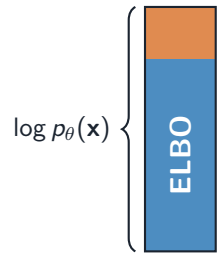
$$\log p_\theta(\mathbf{x}) = \underbrace{\mathbb{E}_{q_\phi}\big[\log p_\theta(\mathbf{x} \mid \mathbf{z})\big] - \mathrm{KL}(q_\phi \,\|\, p(\mathbf{z}))}_{\text{ELBO}} + \underbrace{\mathrm{KL}(q_\phi \,\|\, p_\theta(\mathbf{z} \mid \mathbf{x}))}_{\geq 0}$$

The VAE Loss

Per input $\mathbf{x}$, with one sample $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}$:

$$\mathcal{L}(\mathbf{x}) = \underbrace{-\log p_\theta(\mathbf{x} \mid \mathbf{z})}_{\text{reconstruction (BCE)}} + \underbrace{\tfrac{1}{2} \sum_{k=1}^{d} \left(\mu_k^2 + \sigma_k^2 - \ln \sigma_k^2 - 1\right)}_{\mathrm{KL}(\mathcal{N}(\boldsymbol{\mu}, \mathrm{diag}\, \boldsymbol{\sigma}^2) \,\|\, \mathcal{N}(0, I))}$$

- $\mathcal{L} = -\text{ELBO}$; since $\mathrm{KL} \geq 0$, $-\mathcal{L} \leq \log p_\theta(\mathbf{x})$: minimising $\mathcal{L}$ raises a lower bound on the log-likelihood
- each KL term is 0 only at $\mu_k = 0$, $\sigma_k = 1$
- the two terms pull apart: reconstruction wants small clouds far from each other, the KL pulls every cloud to $\mathcal{N}(0, I)$; the balance: overlapping clouds that fill $\mathcal{N}(0, I)$ without gaps



$d = 16$ VAE, mean over the training digits: $\mathcal{L} = 99.9 = 80.3$ (reconstruction) $+ 19.6$ (KL)

The Reparametrisation Trick

Training needs $\partial\mathcal{L}/\partial\boldsymbol{\mu}$ and $\partial\mathcal{L}/\partial\boldsymbol{\sigma}$, but $\mathbf{z}$ is *drawn at random*: "draw $\mathbf{z}$ from $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2)$" has no derivative with respect to $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$.

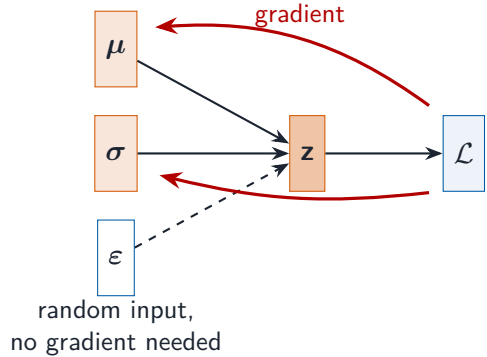
Move the randomness out of the network:

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I)$$

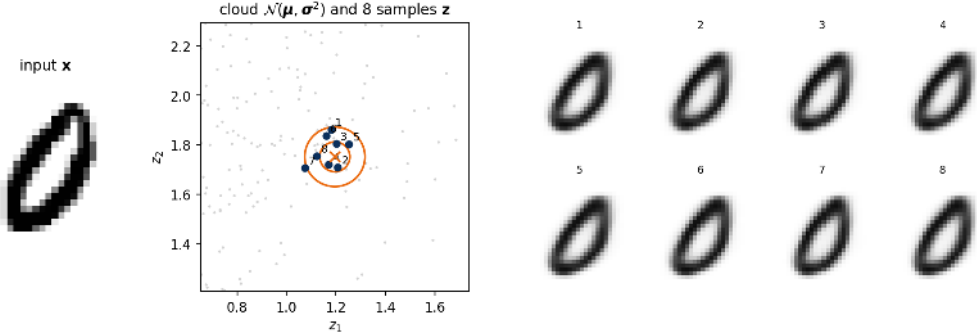
ε is an input that does not depend on the weights; $\mathbf{z}$ is an ordinary function of $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$:

$$\frac{\partial z_k}{\partial \mu_k} = 1, \qquad \frac{\partial z_k}{\partial \sigma_k} = \varepsilon_k,$$

and backpropagation runs from the loss through the decoder, $\mathbf{z}$ and the encoder.



Samples from the Cloud of One Input



One test digit (the 0 that the $d = 2$ VAE rebuilds best; with two latent numbers some unusual digits are rebuilt as another digit). The encoder gives its cloud: centre $\boldsymbol{\mu}$ (cross), widths $\boldsymbol{\sigma}$ (ellipses: 1 and 2σ). Eight noises $\boldsymbol{\varepsilon} \sim \mathcal{N}(0, I)$ give $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}$ (dots); the decoder turns each into a version of the input. In training, one such $\mathbf{z}$ per input is decoded and compared with $\mathbf{x}$; autograd gives $\partial z_1/\partial \mu_1 = 1$, $\partial z_1/\partial \sigma_1 = \varepsilon_1$.

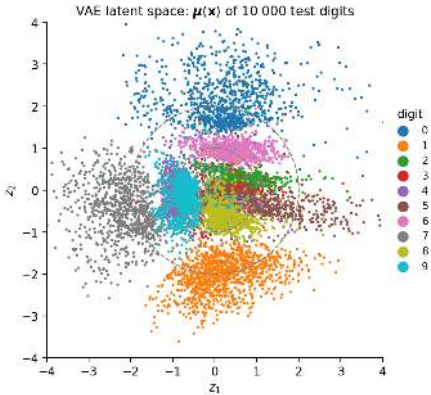
Sampling a VAE

One draw is two random choices, *ancestral sampling* of the model $p_\theta(\mathbf{x}) = \int p_\theta(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})\, d\mathbf{z}$:

$$\mathbf{z} \sim \mathcal{N}(0, I), \qquad \mathbf{x} \sim p_\theta(\mathbf{x} \mid \mathbf{z}) \;\; \text{(each pixel ink with probability } \hat{x}_j(\mathbf{z})\text{)}$$

- in practice we show the *mean image* $\hat{\mathbf{x}}(\mathbf{z}) = G(\mathbf{z})$, the pixel probabilities, rather than one random draw of the pixels: the grey levels are probabilities of ink
- $\mathbf{z}$ carries the content (which digit, its slant and thickness); the pixel draw only adds independent noise
- **reconstruction** (encode $\mathbf{x}$, decode) reproduces a given image; **generation** (draw $\mathbf{z}$, decode) makes a new one
- **interpolation**: encode two digits to $\mathbf{z}_A$, $\mathbf{z}_B$ and decode $(1-t)\,\mathbf{z}_A + t\,\mathbf{z}_B$: one digit turns smoothly into the other, because the latent space is filled without gaps

The Latent Space of a VAE

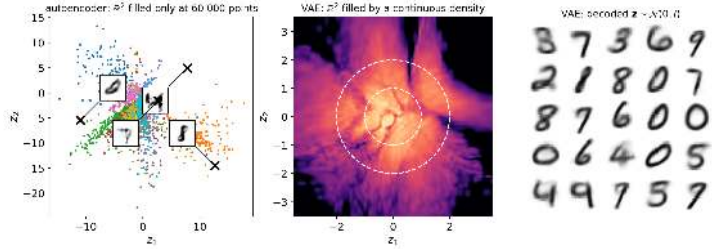


Latent representation: the encoder compresses $\mathbf{x} \in \mathbb{R}^{784}$ into $\mathbf{z} \in \mathbb{R}^d$, coordinates in which similar images are close.

VAE with $d = 2$, trained without labels; each point: the centre $\boldsymbol{\mu}(\mathbf{x})$ of one test digit's cloud, coloured by its label.

- digits of one class gather: the latent space organises the data
- 80% of the centres lie within radius 2 (dashed: radius 1, 2), against 86% for $\mathcal{N}(0, I)$; classes 0, 1, 7 sit at the edge

Codes of the Trained Autoencoder and VAE



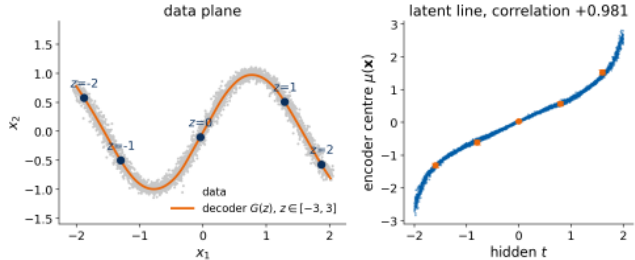
- autoencoder ($d = 2$): 60 000 training codes spread to $|z| \approx 20$; latent points in the gaps ($\times$) decode to images that are no digit
- VAE: the median cloud width is 5.6 times the median spacing of neighbouring codes, so the clouds overlap into a continuous density (dashed: 1 and 2 standard deviations of $\mathcal{N}(0, I)$)
- only 3.6 % of draws $\mathbf{z} \sim \mathcal{N}(0, I)$ land where the density of the clouds is below 10 % of the prior; decoded draws are digits

4

Latent Spaces with Known Causes

JAGIELLONIAN UNIVERSITY
IN KRAKOW

A Curve with One Hidden Number



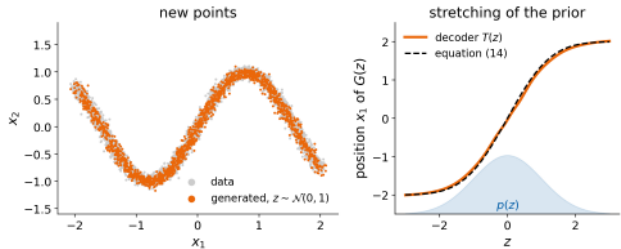
Data: $N = 4000$ points

$$\mathbf{x} = (t,\ \sin 2t) + \text{noise}, \quad t \in [-2, 2]$$

uniform t, Gaussian noise of width 0.05.
One hidden number, two data coordinates.

- VAE with $d = 1$, Gaussian decoder of width 0.05
- the encoder centre μ orders the points like t: rank correlation 0.9996 (linear 0.981: an S-shaped relation)
- clouds narrow: median $\sigma = 0.028$

Generated Points and the Stretching of the Prior



Decoder position $T(z)$ along the curve; a latent interval dz is spread over $|dT/dz|\,dz$, so generated points have density $p(t) = p(z)/|dT/dz|$. A uniform t on $[-2, 2]$ requires

$$T(z) = 4\,\Phi(z) - 2, \qquad \Phi(z) = \int_{-\infty}^{z} \mathcal{N}(z'; 0, 1)\,dz'$$

the inverse of the Gaussian cumulative distribution. The decoder found it by itself: deviation at most 0.12 for $|z| \le 2.5$; 99.2 % of generated points lie in $[-2, 2]$.

The KL Weight on the Curve

β-weighted loss: reconstruction $+ \beta \cdot \mathrm{KL}\big(q_\phi(z|\mathbf{x}) \,\|\, p(z)\big)$

β	median σ	range of μ	correlation of μ with t
0.02	0.0065	-3.99 to 4.04	0.989
1	0.028	-2.83 to 2.81	0.981
20	0.13	-2.20 to 2.19	0.985

- β small: a sharp code, but the centres spread beyond the prior; samples from $\mathcal{N}(0, 1)$ miss the ends of the curve, the autoencoder problem returns
- β large: wide clouds; one output for a whole stretch of the curve is its average, so the decoder cuts the corners of the sine
- the *order* of the latent values stays the same; β changes how finely they resolve t and how well they fill the prior

A Continuous Mixture Model

The VAE is a mixture with a continuous label z:

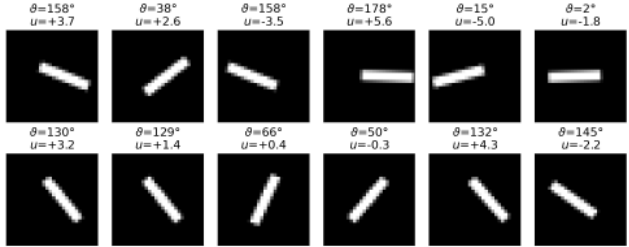
$$p_\theta(\mathbf{x}) = \int p_\theta(\mathbf{x} \mid z)\, p(z)\, dz$$

Mean log-likelihood of 2000 fresh points of the curve:

model	$\langle \log p(\mathbf{x}) \rangle$
mixture of 3 Gaussians	-0.850
mixture of 30 Gaussians	-0.388
VAE, $d = 1$ (integral over z computed numerically)	-0.401
true process (best possible)	-0.360

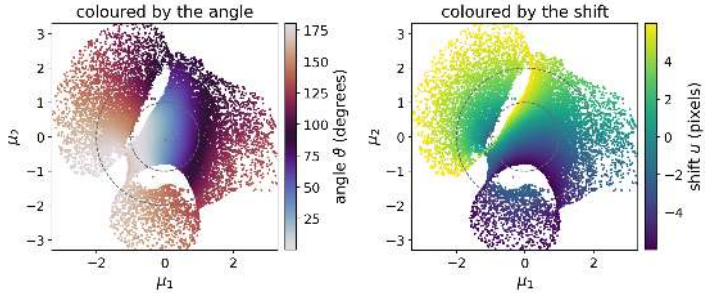
- the 30 component labels are an unordered list; the VAE's z is a coordinate along the curve, defined between the training points
- the ELBO, -0.552 on average, lies below $\log p$ for every point; the gap, 0.151 nats, is the KL divergence between the encoder clouds and the exact posteriors

Bars with an Angle and a Shift



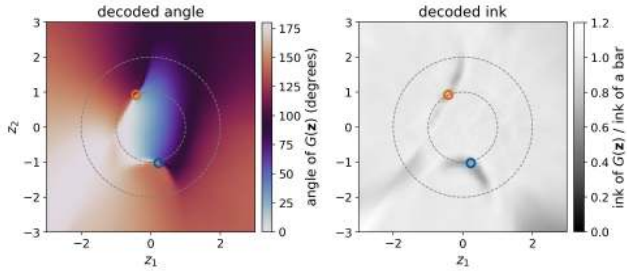
- 20 000 images of 28×28 pixels: one bar of length 16 and width 3 pixels, angle $\vartheta \in [0, \pi)$, horizontal shift $u \in [-6, 6]$ pixels
- 784 grey values fixed by two numbers: the images lie on a 2D surface in $\mathbb{R}^{784}$
- the angle is periodic, $\vartheta = \pi$ is $\vartheta = 0$: the surface is a *cylinder* (a circle of angles times an interval of shifts)
- VAE with $d = 2$, dense $784 \to 256 \to 128 \to 2$, Bernoulli decoder

The Latent Plane of the Bars



- the shift is read off mostly along μ_2: correlation $+0.82$ (μ_1: -0.36)
- the angle does *not* go round a clean ring: the polar angle of $\boldsymbol{\mu}$ follows 2ϑ only weakly (circular correlation 0.47)
- empty slits, and colour jumps across them: where the angle is not continuous

Vortices of the Decoded Angle



Angle of the decoded image $G(\mathbf{z})$ from its second moments (error 0.02° on training bars). Around a small loop, $2\hat{\vartheta}$ turns by a multiple of 2π: the *charge*.

- charge $+1$ near $(-0.42, 0.93)$, charge -1 near $(0.23, -1.02)$ (circles)
- seams start at the cores: faint or split images; latent points decoding to less than 85 % of the ink of a bar carry 5.5 % of the prior probability

Why the Plane Forces Vortices

The data surface is a cylinder; the latent space is a plane.

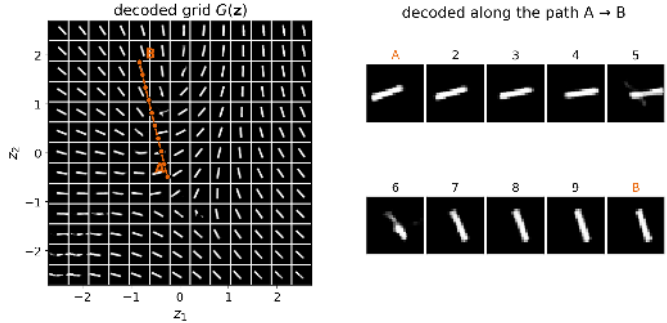
1. if every decoded image were a bar, $2\hat{\vartheta}(\mathbf{z})$ would be a continuous map from the plane to a circle; on a plane without holes such a map turns *zero* times around every closed loop
2. the bars of one shift form a closed loop of images, ϑ from 0 to π; the encoder maps them to a closed latent loop around which the decoded angle turns *once*
3. hence inside that loop the decoder must produce images without a defined angle: a vortex core, net charge $+1$

Arrangement of the Vortices

- topology fixes only the net charge; one vortex with bars on a ring, or a pair of opposite charges joined by seams, are both allowed
- this run: a $+1/-1$ pair near the origin; another random seed can arrange them differently
- the same mathematics as the vortices of the XY model of magnetism, where spin directions play the role of 2ϑ
- the Gaussian prior has no hole, so some prior probability always lands on images that are no proper bar

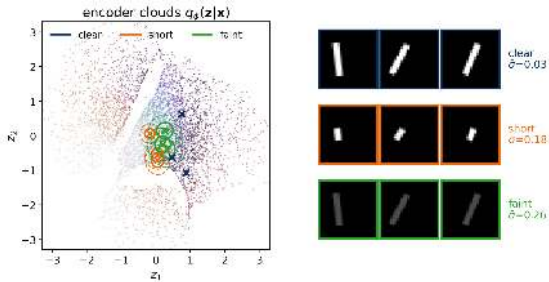
A latent space has a shape. When the data have a periodic cause, a Gaussian latent plane cannot represent them without defects.

A Straight Latent Path Crosses a Seam



From bar A ($\vartheta = 20°$, $u = -4$) to bar B ($\vartheta = 111°$, $u = +4$) on a straight latent line: images 5 and 6 are two crossed bars, because the line passes the $+1$ vortex. A path around the vortex would turn the bar smoothly: a straight latent line is not the natural route between two images.

Encoder Clouds and the Exact Posterior



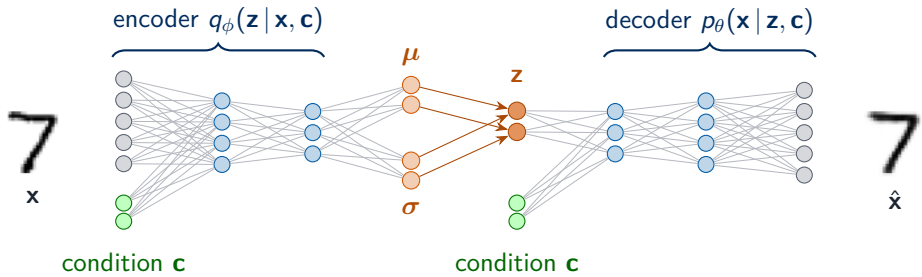
With $d = 2$ the posterior $p_\theta(\mathbf{z}|\mathbf{x}) \propto p_\theta(\mathbf{x}|\mathbf{z})\, p(\mathbf{z})$ can be computed on a grid. Median widths:

image	encoder σ	posterior
clear bar	0.027	0.026
short bar	0.183	0.026
faint bar	0.257	0.032
noisy bar	0.005	0.016

Clear bars: agreement. Short and faint bars (unseen in training): the encoder widens 7–10 times, the posterior hardly. Noisy bars: the encoder is overconfident. The encoder only *approximates* the posterior.

5
Conditional Generation

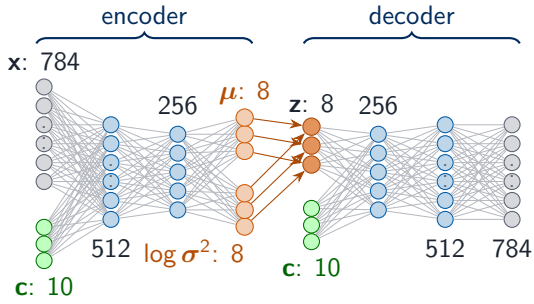
Conditional VAE



$\mathbf{c}$ is an extra input of *both* networks, $q_\phi(\mathbf{z} \mid \mathbf{x}, \mathbf{c})$ and $p_\theta(\mathbf{x} \mid \mathbf{z}, \mathbf{c})$; the model is $p_\theta(\mathbf{x} \mid \mathbf{c}) = \int p_\theta(\mathbf{x} \mid \mathbf{z}, \mathbf{c})\, p(\mathbf{z})\, d\mathbf{z}$, with the same loss as the VAE. To generate: fix $\mathbf{c}$, draw $\mathbf{z} \sim \mathcal{N}(0, I)$, compute $G(\mathbf{z}, \mathbf{c})$.

K. Sohn, H. Lee, X. Yan, *Learning Structured Output Representation using Deep Conditional Generative Models*, NeurIPS 2015, NeurIPS proceedings

Conditional VAE Layers and Parameters



layer	output	param.
encoder, input $[\mathbf{x}, \mathbf{c}]$: $784 + 10$		
dense + ReLU	512	407 040
dense + ReLU	256	131 328
dense $\to \boldsymbol{\mu}$	8	2 056
dense $\to \log \boldsymbol{\sigma}^2$	8	2 056
decoder, input $[\mathbf{z}, \mathbf{c}]$: $8 + 10$		
dense + ReLU	256	4 864
dense + ReLU	512	131 584
dense (logits)	784	402 192

$\mathbf{c}$: the one-hot digit, appended to the input of both networks; $d = 8$.
Total 1 081 120 parameters, all trainable (torchinfo).

Condition and Latent Vector

- $\mathbf{c}$ tells the decoder *what* to draw. Every bit stored in $\mathbf{z}$ costs KL, while what $\mathbf{c}$ supplies is free, so $\mathbf{z}$ keeps only what $\mathbf{c}$ does not say: the *style* (slant, thickness, size)

- a class k enters *one-hot*, $\mathbf{e}_k = (0, \ldots, 0, 1, 0, \ldots, 0)$ with the 1 at position k, not as the number k, which would impose an order on the classes; physical parameters such as (T, h) enter as numbers

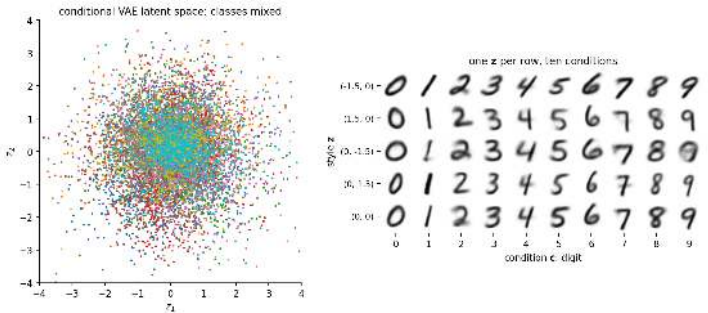
Sampling a Conditional VAE

The model is $p_\theta(\mathbf{x} \mid \mathbf{c}) = \int p_\theta(\mathbf{x} \mid \mathbf{z}, \mathbf{c})\, p(\mathbf{z})\, d\mathbf{z}$. Sampling: we *choose* $\mathbf{c}$ and *draw* $\mathbf{z}$:

$$\mathbf{c} \text{ given (what)}, \qquad \mathbf{z} \sim \mathcal{N}(0, I) \text{ (how)}, \qquad \hat{\mathbf{x}} = G(\mathbf{z}, \mathbf{c})$$

- one $\mathbf{c}$, many $\mathbf{z}$: many instances of the same class, different in style (digits 7 in many handwritings; cells of one type)
- one $\mathbf{z}$, many $\mathbf{c}$: the same style for every class (the rows of the digit grid on the next slides keep slant and thickness)
- continuous conditions work the same way: fix the temperature T and draw $\mathbf{z}$ to get spin configurations at that T
- the condition must be known for every training image: a conditional VAE needs labels; what the labels do not describe is left to $\mathbf{z}$

Latent Space of the Conditional VAE



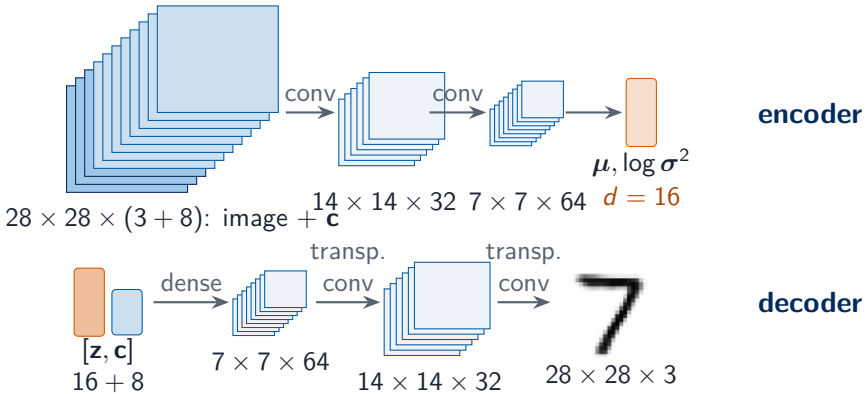
Conditional VAE with $d = 2$, label given as $\mathbf{c}$. Left: the centres $\boldsymbol{\mu}(\mathbf{x}, \mathbf{c})$ of the test digits, coloured by digit, are mixed: the position no longer tells the class. Right: one fixed $\mathbf{z}$ per row, the ten conditions along the row: the digit changes, slant and thickness stay. **c chooses what, z chooses how.**

Conditional VAE on MNIST



- each row: $\mathbf{c}$ = one digit, 0 to 9; each column: one random $\mathbf{z} \sim \mathcal{N}(0, I)$, the *same* in every row
- along a column the style stays and the digit changes: $\mathbf{z}$ carries the style, $\mathbf{c}$ the identity
- the generated digits are checked with a classifier of real digits

The Condition in a Convolutional Network



BloodMNIST: the one-hot cell type enters as 8 extra constant channels (1 for the requested type, 0 otherwise); stride-2 convolutions halve the size, transposed convolutions double it.

Layers of the Convolutional Conditional VAE

layer	output	param.
encoder, input: image and $\mathbf{c}$ *as channels,* $28 \times 28 \times (3+8)$		
conv 3×3, stride 2, ReLU	$14 \times 14 \times 32$	3 200
conv 3×3, stride 2, ReLU, flatten	$7 \times 7 \times 64$	18 496
dense $\to \boldsymbol{\mu}$	16	50 192
dense $\to \log \boldsymbol{\sigma}^2$	16	50 192
decoder, input $[\mathbf{z}, \mathbf{c}]$*:* $16+8$		
dense, ReLU, reshape	$7 \times 7 \times 64$	78 400
transposed conv 3×3, stride 2, ReLU	$14 \times 14 \times 32$	18 464
transposed conv 3×3, stride 2 (logits)	$28 \times 28 \times 3$	867

Shapes: height $\times$ width $\times$ channels; $\mathbf{c}$: the one-hot cell type (8 numbers); $d = 16$. A 3×3 convolution from C_{in} to C_{out} channels has $9\,C_{\text{in}}C_{\text{out}} + C_{\text{out}}$ parameters.

Total 219 811 parameters, all trainable (torchinfo).

Generating a Chosen Digit

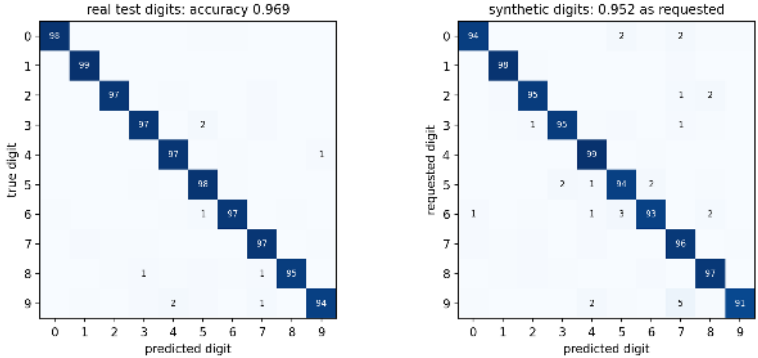
Only the decoder: condition $\mathbf{c} = \mathbf{e}_k$ (one-hot digit k), random style $\mathbf{z} \sim \mathcal{N}(0, I)$, image sigmoid $\big(G(\mathbf{z}, \mathbf{c})\big)$.

```
def generate_digit(k, n=10):
    c = F.one_hot(torch.full((n,), k),
                  10).float()
    z = torch.randn(n, cvae.d)
    with torch.no_grad():
        return torch.sigmoid(
            cvae.decode(z, c))
```

Row k: `generate_digit(k, 12)`; each column a new $\mathbf{z}$.

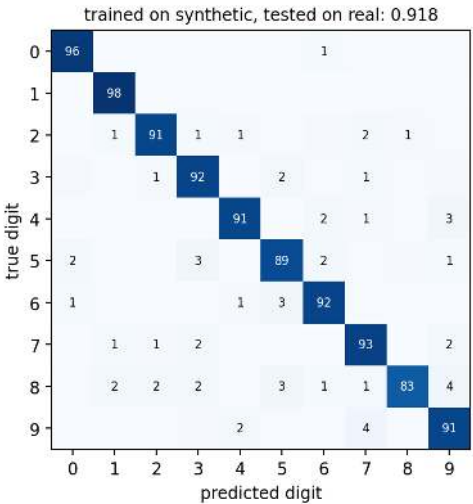


Classifier Check of Synthetic Digits



Classifier $784 \to 256 \to 10$, trained on real digits only. Left: on the real test digits (row: true digit). Right: on 1000 generated digits per class (row: requested digit); entries in % of the row. Diagonal mean: 0.95; the confusions resemble those of real digits.

Training on Synthetic Digits Only



The same classifier, trained on 60 000 *generated* digits (6000 per class, labels = requested digits), tested on the real test digits:

- trained on real digits: accuracy 0.969
- trained on synthetic digits: accuracy 0.918

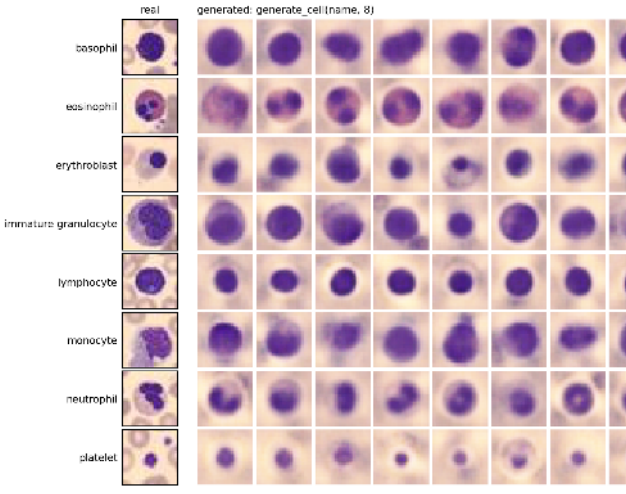
The gap measures what the generator misses of the real distribution: styles it never produces, and blurred strokes.

Generating a Cell of a Chosen Type

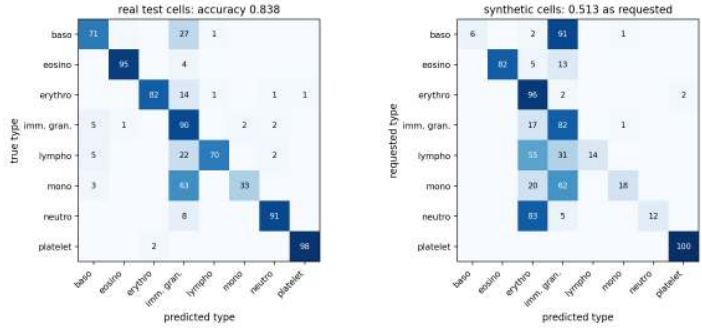
The same recipe, with the cell type as condition and an RGB output:

```
def generate_cell(name, n=8):
    k = names.index(name)
    c = F.one_hot(torch.full((n,),
                  8).float()
    z = torch.randn(n, bcvae.d)
    with torch.no_grad():
        return torch.sigmoid(
            bcvae.decode(z, c))
```

First column: one real cell of the type; then 8 generated.

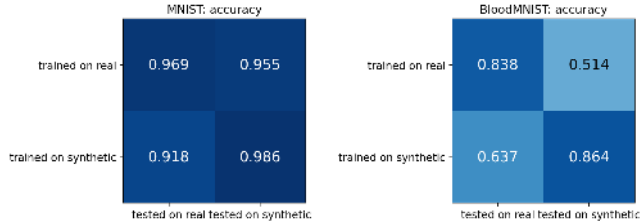


Classifier Check of Synthetic Cells



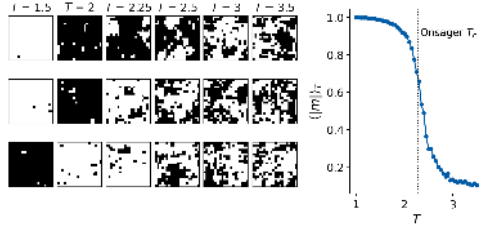
CNN trained on real cells only. Left: real test cells (accuracy 0.84). Right: 500 generated cells per requested type (diagonal mean 0.51). Platelets (100%) and erythroblasts (96%), told apart by size, come out right; generated basophils are taken for immature granulocytes (91%), neutrophils for erythroblasts (83%): the VAE blurs the shape of the nucleus.

Training and Testing on Real and Synthetic Data



- **trained on real, tested on synthetic:** does the generator draw the requested class?
- **trained on synthetic, tested on real:** do the synthetic data carry what is needed to recognise real instances?
- MNIST: synthetic digits almost replace real ones (0.92 against 0.97); blood cells: trained on synthetic, 0.64 on real cells (0.84 with real training) but 0.86 on synthetic: types easy to tell apart, but not the real ones

Ising Configurations at a Chosen Temperature



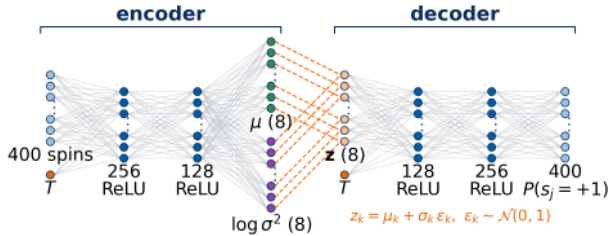
Monte Carlo configurations (white: up, black: down) and $\langle |m| \rangle_T$; dotted: Onsager's T_c

The 2D Ising model of Lecture 4, 20×20 spins $s_i = \pm 1$:

$$p(\mathbf{s} \mid T) = \frac{e^{-H(\mathbf{s})/T}}{Z(T)}, \qquad H = -\sum_{\langle ij \rangle} s_i s_j$$

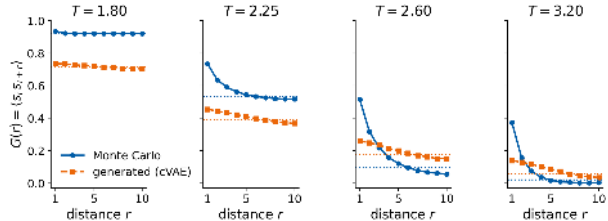
- data: 10 200 Metropolis configurations, 200 at each of 51 temperatures
- task: a conditional VAE generating configurations for a *requested* T
- test: does it reproduce the physics?

Ising Conditional VAE: Layers



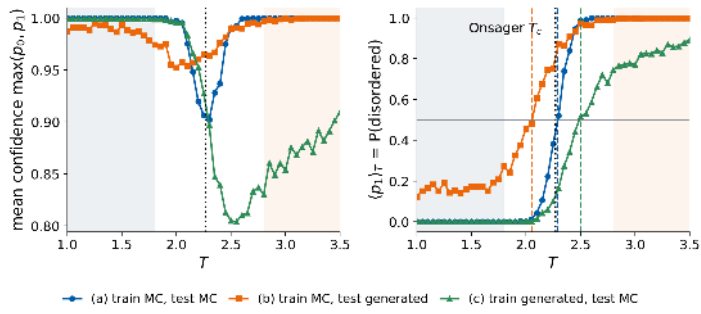
- pixels $x_i = (s_i + 1)/2 \in \{0, 1\}$, $D = 400$; condition $c = (T - 2.25)/0.75$ enters the encoder and the decoder
- encoder $401 \to 256 \to 128 \to (\boldsymbol{\mu}, \log \boldsymbol{\sigma}^2)$, latent dimension 8; decoder $9 \to 128 \to 256 \to 400$ Bernoulli logits
- parameters (torchinfo): encoder 137 872, decoder 137 104, total 274 976, all trainable

Generated Configurations against Monte Carlo



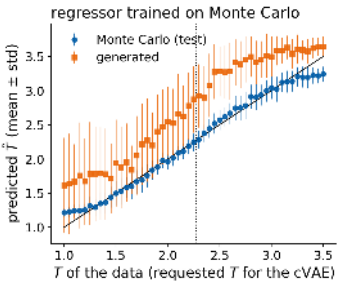
- correlation $G(r) = \langle s_i s_{i+r} \rangle$: generated spins are less correlated at short range and too correlated far apart (dotted: $\langle m \rangle^2$)
- $\langle |m| \rangle$: largest gap 0.234, at $T = 2.10$; energy per spin too high at *every* T, largest gap 0.634 at $T = 2.30$
- at $T \leq 1.5$, 8.5 % of generated configurations have $|m| < 0.5$ (Monte Carlo: none)

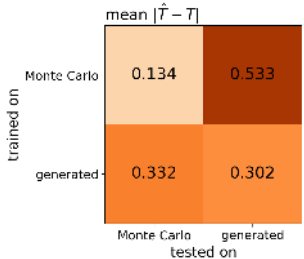
Physics Checks with a Phase Classifier



CNN classifier ordered/disordered, trained on $T < 1.8$ and $T > 2.8$ (shaded), T_c read where $\langle p_1 \rangle_T$ crosses $1/2$ (3 seeds): (a) MC $\to$ MC: 2.292 ± 0.012; (b) MC $\to$ generated: 2.056 ± 0.012; (c) generated $\to$ MC: 2.501 ± 0.057; Onsager: 2.269

Temperature Read Back from Generated Configurations





A CNN regressor predicts T from a configuration (mean absolute error $|\hat{T} - T|$, one seed per model).

- trained and tested on Monte Carlo: 0.134
- tested on generated: 0.533; they look *hotter* than requested, by up to 0.78 at $T = 2.5$

The cVAE learns the trend with T, not the exact Boltzmann distribution.

6

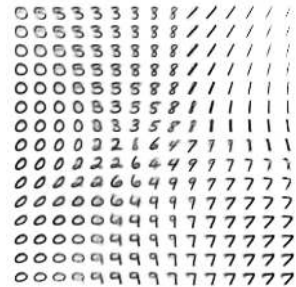
Disentangled Latent Features

JAGIELLONIAN UNIVERSITY
IN KRAKOW

Latent Coordinates and Features

Each latent coordinate z_k is a direction; moving along it, the others fixed, changes the decoded image (a traversal).

- in the $d = 2$ VAE (right) moving z_1 or z_2 changes several properties at once: the digit identity, its slant, its width
- **disentangled**: each coordinate responds to one independent property of the data and ignores the others
- the goal: one latent node $\leftrightarrow$ one distinct feature, found without labels (unlike the condition $\mathbf{c}$ of a cVAE)



$d = 2$ VAE: decoded images on a grid of (z_1, z_2)

The β-VAE Loss

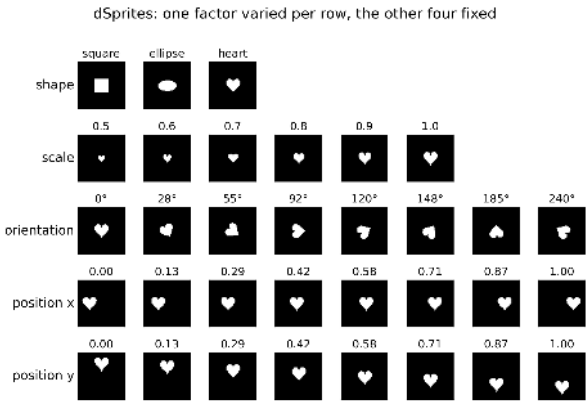
One weight more in the VAE loss:

$$\mathcal{L}_\beta(\mathbf{x}) = \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\big[-\log p_\theta(\mathbf{x}\,|\,\mathbf{z})\big]}_{\text{reconstruction}} + \beta\,\underbrace{\mathrm{KL}\big(q_\phi(\mathbf{z}\,|\,\mathbf{x})\,\|\,\mathcal{N}(0,I)\big)}_{\sum_k \frac{1}{2}(\mu_k^2+\sigma_k^2-\ln\sigma_k^2-1)}, \qquad \beta \geq 1$$

- $\beta = 1$: the VAE, minus the ELBO; $\beta > 1$: information stored in $\mathbf{z}$ costs more
- the prior $\mathcal{N}(0, I)$ has *independent* coordinates; a stronger pull towards it makes each kept coordinate carry one independent factor of variation
- a coordinate with $\mu_k \approx 0$, $\sigma_k \approx 1$ for every input (KL ≈ 0) carries nothing: the KL per coordinate shows which are used
- the price: blurrier reconstructions and samples; β trades sharpness for interpretable coordinates

I. Higgins et al., β-VAE, ICLR 2017, openreview.net/forum?id=Sy2fzU9gl

The dSprites Data



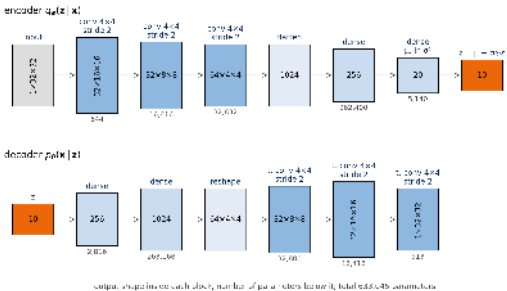
737 280 images of one sprite, every combination of five known factors once:

- shape (3), scale (6), orientation (40)
- position x and y (32 each)

Known factors: every latent coordinate can be checked against them. Here 100 000 training images at 32×32 pixels.

L. Matthey, I. Higgins, D. Hassabis, A. Lerchner, dSprites (2017), github.com/deepmind/dsprites-dataset, Apache 2.0

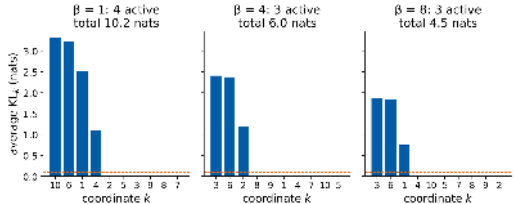
β-VAE Layers and Parameters



layer	output	param.
encoder, input $1 \times 32 \times 32$		
3 conv 4×4, stride 2	$64 \times 4 \times 4$	49 792
dense	256	262 400
dense $\to (\boldsymbol{\mu}, \log \boldsymbol{\sigma}^2)$	2×10	5 140
decoder, input $\mathbf{z}$*: 10*		
2 dense, reshape	$64 \times 4 \times 4$	265 984
3 transposed conv	$1 \times 32 \times 32$	49 729

dSprites, 32×32 pixels; shapes: channels $\times$ height $\times$ width; ReLU between layers. Total 633 045 parameters, all trainable (torchinfo).

Information per Latent Coordinate



Average KL_k of each of the 10 coordinates over the test images (dashed: 0.1 nats, the threshold for an *active* coordinate):

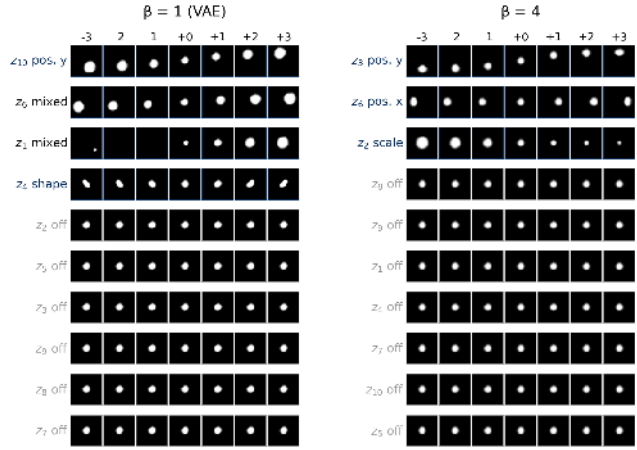
	$\beta = 1$	$\beta = 4$	$\beta = 8$
active coordinates	4	3	3
total KL (nats per image)	10.15	5.98	4.46
reconstruction (nats per image)	20.56	26.79	30.57

A larger β stores less information and reconstructs worse; the unused coordinates stay at the prior.

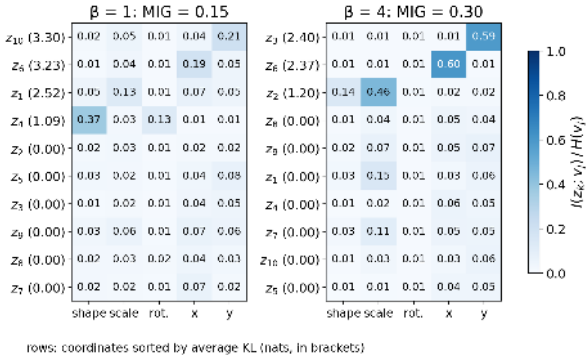
Sampling a β-VAE

Traversal: encode one image, move one coordinate z_k from -3 to $+3$, decode.

- $\beta = 4$: one coordinate each for position y, position x, scale; the rest off
- $\beta = 1$: position y separate, the rest mixed
- no labels: properties are found, then named by looking



Measuring Disentanglement



Matrix: $I(z_k; v_j)/H(v_j)$, coordinate z_k, factor v_j. **Mutual information gap**:

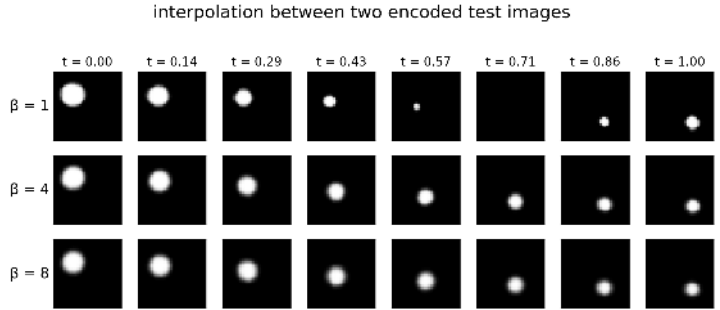
$$\mathrm{MIG} = \frac{1}{J}\sum_{j=1}^{J} \frac{I(z_{(1)}; v_j) - I(z_{(2)}; v_j)}{H(v_j)}$$

$z_{(1)}, z_{(2)}$: the two most informative coordinates.

- MIG 0.15, 0.30, 0.30 for $\beta = 1, 4, 8$
- orientation: learnt by none; shape: weakly

R. T. Q. Chen et al., NeurIPS 2018, arXiv:1802.04942

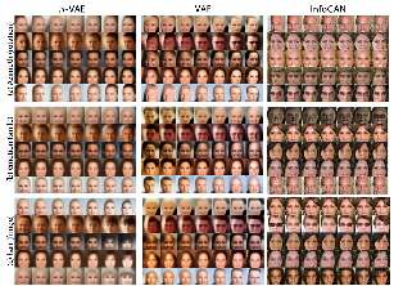
Interpolation between Two Images



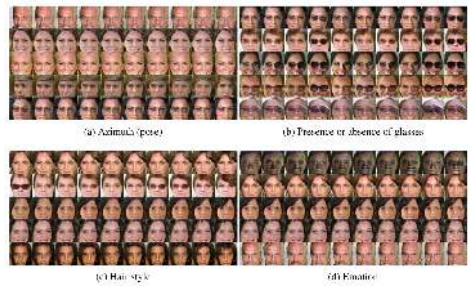
Encode two test images, decode the straight line $(1-t)\,\boldsymbol{\mu}_A + t\,\boldsymbol{\mu}_B$:

- $\beta = 1$: the sprite shrinks and vanishes on the way (empty canvas at $t = 0.71$) before reappearing
- $\beta = 4, 8$: the sprite moves and changes size smoothly, because position and size have their own coordinates

Disentanglement on Faces in the Literature



Higgins et al., ICLR 2017, Fig. 1: traversals on CelebA faces; columns β-VAE ($\beta = 250$), VAE, InfoGAN; rows: rotation, smile, fringe



Chen et al., NIPS 2016, Fig. 6: InfoGAN codes for pose, glasses, hair style, emotion

On faces, rotation and smile get separate coordinates with large β; the plain VAE mixes rotation with smile, glasses and gender.

I. Higgins et al., ICLR 2017, openreview.net/forum?id=Sy2fzU9gl; X. Chen et al., NIPS 2016, arXiv:1606.03657

VAE, Conditional VAE and β-VAE

	VAE	conditional VAE	β-VAE
trained on	images $\mathbf{x}$	pairs $(\mathbf{x}, \mathbf{c})$	images $\mathbf{x}$
loss	reconstruction + KL	reconstruction + KL	reconstruction + β KL
to sample	$\mathbf{z} \sim \mathcal{N}(0, I)$	choose $\mathbf{c}$, $\mathbf{z} \sim \mathcal{N}(0, I)$	$\mathbf{z} \sim \mathcal{N}(0, I)$, or move one z_k
$\mathbf{z}$ controls	everything, mixed	what $\mathbf{c}$ does not say (style)	separate properties, one per coordinate
labels	not needed	needed	not needed
use	new samples	samples of a chosen class or T	finding and controlling factors

Summary

- a **generative model** learns the distribution $p(\mathbf{x})$ of the data from samples, and draws new samples from it
- **maximum likelihood**: choose the parameters that make the observed data most probable; it minimises the KL divergence to the data
- **VAE**: a latent space with a known distribution, $\mathcal{N}(0, I)$; trained on reconstruction + KL; decode random $\mathbf{z}$ to generate
- **conditional VAE**: generates what we ask for, a digit or a cell type
- β-**VAE**: one latent coordinate per feature, found without labels: a tool for interpretation

Next Lecture: Decision Trees and Ensembles

- data as tables: one row per sample, one column per measured feature
- decision trees: splits chosen by the Gini impurity or the entropy, rules that can be read
- random forests: many trees grown on resampled data, their votes combined
- boosting: trees added one at a time, each correcting the errors of the previous ones

Thank You!

Questions, project ideas, feedback: marcin.plodzien@uj.edu.pl

Website · GitHub · Google Scholar



